Laboratório 2 — Automação de Deploys com CI/CD (GitHub Actions)

Apostila de apoio — Análise e Desenvolvimento de Sistemas

1. Visão geral do laboratório

1.1 O que você vai fazer

Neste laboratório você vai:

1. Entender o que são **CI** (Integração Contínua) e **CD** (Entrega/Implantação Contínua) e por que eles existem.
2. Enviar para o GitHub o projeto Terraform que você construiu no Lab 1 (`main.tf` e a pasta `site/`).
3. Escrever um arquivo de **workflow** do GitHub Actions que executa, automaticamente, o mesmo ciclo `init → fmt → validate → plan → apply` que você digitou à mão no Lab 1.
4. Comparar o comportamento da pipeline em um **Pull Request** (que só mostra o plano) com o comportamento em um `push` na branch `main` (que efetivamente aplica a mudança).
5. Provocar de propósito uma falha de formatação e um rollback, para sentir na prática por que uma esteira de CI/CD reduz o risco de cada mudança.

> **Cenário:** você continua no papel de Administrador(a) de Ambientes da StackUp Ltda. — agora a tarefa é automatizar o deploy do ambiente que você montou no Lab 1. Veja o documento "Instruções Gerais" para o cenário completo e as regras de uso de IA como ferramenta de trabalho.

> **Trabalho em dupla:** continue com a mesma dupla e o mesmo repositório do Lab 1. Troque de papel em relação ao Lab 1 — quem só observou agora digita os comandos `git`/`terraform` e edita o `terraform.yml` no VS Code, e vice-versa. As respostas escritas dos exercícios (seção 5) e a contribuição para o entregável final (seção 7) continuam sendo individuais, mesmo com repositório e pipeline compartilhados pela dupla.

1.2 Objetivo de aprendizagem

Ao final deste laboratório, você deve ser capaz de explicar e demonstrar:

- A diferença entre CI e CD, e por que os dois normalmente andam juntos.
- Por que a etapa que aplica mudanças (`apply`) deve rodar em um servidor da pipeline, e não na máquina de cada desenvolvedor.
- Por que se separa "propor uma mudança" (`plan`, em um Pull Request) de "aplicar uma mudança" (`apply`, só depois do merge em `main`).
- Por que uma pipeline deve **falhar cedo** (*fail fast*) em vez de deixar um erro pequeno passar despercebido até um estágio mais avançado e mais caro de corrigir.

1.3 Onde este laboratório se encaixa

```
[1] Provisionar        →   [2] Automatizar        →   [3] Observar e reagir
    a infraestrutura           o deploy                   a incidentes
    (Terraform/Docker)         (CI/CD)                    (Prometheus/Grafana)
                               ▲ você está aqui
```

No Lab 1 você digitou `terraform apply` com as próprias mãos, sempre que quis mudar algo. Neste Lab 2, esse mesmo comando deixa de ser digitado por você e passa a ser executado automaticamente, sempre da mesma forma e com registro de auditoria, por uma pipeline disparada a cada mudança no repositório. É esse deslocamento — de "eu aplico manualmente" para "a esteira aplica por mim, de forma consistente" — que caracteriza a automação de deploys em DevOps.

2. O que é CI/CD

- **CI (Integração Contínua):** toda mudança de código é automaticamente testada, validada e integrada ao repositório principal, com feedback rápido sobre erros.
- **CD (Entrega/Implantação Contínua):** toda mudança validada é automaticamente entregue (ou implantada) em um ambiente, sem intervenção manual repetitiva.

No contexto de Infraestrutura como Código, isso significa: em vez de você digitar `terraform apply` na sua máquina, é uma **pipeline** — rodando em um servidor do GitHub — que executa esse ciclo por você, sempre da mesma forma, de forma auditável.

> **Por que isso importa mesmo em um projeto pequeno como o do Lab 1?** Porque o valor do CI/CD não está na complexidade do projeto, e sim na **consistência**: uma pessoa cansada, apressada ou distraída pode esquecer um passo do ciclo `init/plan/apply`; uma pipeline nunca esquece. Quanto mais cedo você se acostuma a delegar essa disciplina a uma esteira automatizada, menos disso vira hábito manual — e hábito manual é, historicamente, a maior fonte de erro humano em produção.

3. Preparando o repositório

Passo 0 — Antes de começar: identifique-se para o Git e prepare o GitHub

Este passo só precisa ser feito **uma vez por máquina** (se você já fez isso em outra aula, pode pular). Ele resolve dois problemas que travam quase todo mundo na primeira vez que usa Git: o programa não sabe "quem é você" e o GitHub não aceita mais senha comum para receber arquivos.

O que é o Git, rapidamente: é o programa (instalado no seu computador) que guarda o histórico de mudanças dos seus arquivos. O **GitHub** é o site que guarda uma cópia desse histórico na internet, para a dupla compartilhar o mesmo projeto. `git` é o comando que você digita no terminal; GitHub é o site.

0.1 — Diga ao Git o seu nome e e-mail

Cada vez que você salva uma mudança (um *commit*), o Git registra quem fez aquilo. Sem essa informação configurada, o comando `git commit` do Passo 1 vai falhar com o erro `Please tell me who you are`. Configure uma única vez, digitando (troque pelo seu nome e um e-mail válido — pode ser o mesmo da sua conta do GitHub):

```
git config --global user.name "Seu Nome"

git config --global user.email "seu-email@exemplo.com"
```

0.2 — Crie o repositório vazio no site do GitHub

O comando `git remote add origin` do Passo 1 só funciona se esse endereço já existir no GitHub — o Git não cria o repositório lá sozinho, ele só *aponta* para um que já existe. Então, antes de digitar qualquer comando, crie-o pela interface do site:

1. Acesse github.com e faça login.
2. No canto superior direito, clique no ícone **+** → **New repository**.
3. Em *Repository name*, digite `meu-projeto` (o mesmo nome usado nos comandos abaixo).
4. Deixe como **Public** (ou Private, tanto faz para o laboratório).
5. **Importante:** não marque nenhuma das opções "Add a README file", "Add .gitignore" ou "Choose a license". Se marcar, o GitHub cria arquivos lá que vão conflitar com o histórico que você já tem na sua máquina, e o `git push` do Passo 1 vai falhar.
6. Clique em **Create repository**.

A página que abre em seguida já mostra o endereço `https://github.com/SEU_USUARIO/meu-projeto.git` que você vai usar no comando `git remote add origin` — copie o seu usuário real no lugar de `SEU_USUARIO`.

0.3 — Autentique-se para poder enviar (`push`) arquivos

O que é "autenticar-se": é provar para o GitHub que é realmente você tentando enviar arquivos, e não outra pessoa. Até 2021 bastava digitar a senha da sua conta quando o Git pedia — hoje o GitHub **não aceita mais isso** por HTTPS. No lugar da senha, você usa um **token de acesso pessoal (PAT)**, que é como uma senha longa e temporária, criada só para isso.

Como criar o seu token:

1. No GitHub, clique na sua foto (canto superior direito) → **Settings**.
2. No menu à esquerda, role até **Developer settings** (é o último item).
3. Clique em **Personal access tokens** → **Tokens (classic)** → **Generate new token** → **Generate new token (classic)**.
4. Dê um nome (ex.: `laboratorio-devops`), marque a caixa **repo** (dá permissão para enviar código), e clique em **Generate token**.
5. O GitHub mostra o token **uma única vez**. Copie e guarde num lugar seguro (um bloco de notas, por exemplo) — se perder, terá que gerar outro.

Quando, no Passo 1, o comando `git push -u origin main` pedir usuário e senha no terminal:

- **Username:** seu usuário do GitHub (o normal, de login).
- **Password: cole o token** que você acabou de gerar — não é a senha da sua conta.

> **Dica para quem está na máquina virtual (VirtualBox):** para colar o token (ou qualquer bloco de texto longo, como o `terraform.yml` mais à frente) dentro da VM, o recurso de **área de transferência compartilhada** (*shared clipboard*) precisa estar ativado. No menu da janela da VirtualBox: **Dispositivos**

> → **Área de Transferência Compartilhada** → **Bidirecional**. Se essa opção não aparecer ou não funcionar, é sinal de que as **Guest Additions** (um pacote de drivers extra do VirtualBox) não estão instaladas na VM — sem elas, colar textos longos do computador real para dentro da VM não funciona, e digitar um YAML inteiro à mão é fonte garantida de erro de indentação.

Passo 1 — Suba a pasta do Lab 1 (`main.tf`, `site/`) para um repositório no GitHub:

No terminal integrado do VS Code (ainda com a pasta `meu-projeto` do Lab 1 aberta), digite — um comando por vez:

```
cd ~/meu-projeto

git init

git add .

git commit -m "Infraestrutura inicial com Terraform"

git branch -M main

git remote add origin https://github.com/SEU_USUARIO/meu-projeto.git

git push -u origin main
```

Quando o terminal pedir usuário e senha, use as credenciais do Passo 0.3 (usuário normal + token no lugar da senha).

> **Por que o Git entra aqui, e não só o Terraform?** O Git é o que dá à sua infraestrutura um **histórico versionado** — cada mudança no `main.tf` fica registrada, com autor e data, e pode ser revertida. O GitHub Actions, por sua vez, é o que "ouve" esse repositório: toda vez que algo muda nele, a pipeline acorda e reage. Sem o Git, não haveria o que a pipeline observasse.

Passo 2 — Crie o arquivo de workflow em `.github/workflows/terraform.yml`:

No VS Code (Explorer, mesma pasta `meu-projeto`):

1. Clique com o botão direito na raiz do projeto → **New Folder** → nomeie `.github`.
2. Clique com o botão direito em `.github` → **New Folder** → nomeie `workflows`.
3. Clique com o botão direito em `workflows` → **New File** → nomeie `terraform.yml`.
4. Cole o conteúdo abaixo e salve com `Ctrl+S`.

`terraform.yml`:

```
name: Terraform CI/CD

on:
  push:
    branches: [main]
  pull_request:
```

```
    branches: [main]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Instalar Terraform
        uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init

      - name: Terraform Format Check
        run: terraform fmt -check

      - name: Terraform Validate
        run: terraform validate

      - name: Terraform Plan
        run: terraform plan

      - name: Terraform Apply (somente na branch main)
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: terraform apply -auto-approve

      - name: Smoke test (verificação pós-deploy)
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: |
          sleep 5
          docker ps
```

> Os runners `ubuntu-latest` do GitHub Actions já vêm com Docker pré-instalado, então o `docker_provider` do Terraform funciona diretamente — os containers sobem dentro do próprio runner (efêmero: eles somem quando a execução termina).

Etapa	Por que ela existe
`actions/checkout@v4`	Sem isso, o runner começaria vazio — essa etapa traz o código do repositório (incluindo `main.tf` e `site/`) para dentro dele
`hashicorp/setup-terraform@v3`	Instala o Terraform de forma padronizada no runner, na mesma versão para toda a equipe, sem depender de configuração manual
`terraform fmt -check`	Verifica a formatação **antes** de gastar tempo com `validate`/`plan` — um erro de estilo é barato de detectar cedo e caro de rastrear tarde
`terraform validate`	Confere a sintaxe e a coerência interna do `.tf`, sem tocar em nenhum recurso real ainda

Etapa	Por que ela existe
`terraform plan`	Roda em **toda** execução (push ou PR): mostra o que mudaria, mesmo quando a mudança não será aplicada
`terraform apply -auto-approve`	Só roda quando a condição `if` é satisfeita — é o único passo que de fato altera a infraestrutura
Smoke test (`docker ps`)	Confirma, de forma simples e rápida, que o `apply` realmente deixou algo no ar — não basta o comando "ter rodado sem erro"

Passo 3 — Envie o workflow:

No terminal integrado, digite — um comando por vez:

```
git add .github/workflows/terraform.yml

git commit -m "Adiciona pipeline de CI/CD"

git push
```

Passo 4 — Acompanhe a execução na aba Actions do repositório no GitHub.

> **Nota sobre o banco de dados do Lab 1:** se o seu `main.tf` já incluir o `docker_container.db` e o `docker_volume.db_data` do Lab 1, a pipeline também vai recriá-los a cada execução — mas, como o runner do GitHub Actions é **efêmero** (destruído ao fim de cada execução), qualquer dado que você tenha inserido via `psql` na seção 6 do Lab 1 **não sobrevive** de uma execução para a outra: o volume nasce e morre junto com o runner. Isso é uma boa demonstração prática de que persistência de dados real, em CI/CD, depende de um banco **externo** ao runner — não de um container efêmero rodando dentro dele.

4. Por que `apply` só roda na branch `main`?

Repare na condição `if: github.ref == 'refs/heads/main'`. Isso separa duas responsabilidades:

- Em um **Pull Request**, a pipeline roda `plan` (mostra o que mudaria) mas **não aplica** — dando a um revisor humano a chance de ver o impacto antes de aprovar.
- Só quando o código é **mesclado na** `main` é que a mudança é de fato aplicada.

Essa separação entre "propor mudança" e "aplicar mudança" é um dos princípios centrais de CI/CD: reduzir o risco de cada alteração em produção.

5. Exercícios de prática — Lab 2

1. **Primeira execução:** dispare a pipeline com um `push` simples (ex.: alterar um comentário no `main.tf`) e acompanhe cada etapa (`init`, `fmt`, `validate`, `plan`, `apply`) na aba Actions.

2. **Pull Request como gate:** crie uma branch `feature/nova-porta`, altere a porta externa do container, e abra um Pull Request para `main`. Confirme que a pipeline executa até o `plan` mas **não** aplica. Só depois faça o merge e observe a aplicação automática.

O que é uma "branch": é uma cópia paralela do seu código, onde você pode mexer sem afetar a versão que já está funcionando na `main`. *O que é um "Pull Request" (PR):* é um pedido, feito pelo site do GitHub, para juntar (mesclar) essa cópia paralela de volta na `main` — é o momento em que alguém (a dupla, ou o professor) pode revisar a mudança antes dela valer de verdade.

No terminal, digite — um comando por vez:

```
git checkout -b feature/nova-porta
```

Edite o `main.tf` no VS Code, mudando a porta externa do container (por exemplo, de `8080` para `8081`) e salve.

```
git add main.tf

git commit -m "Altera porta externa do container"

git push -u origin feature/nova-porta
```

O `-u` na primeira vez que você envia uma branch nova diz ao Git para lembrar, dali em diante, para onde essa branch deve ir — nos próximos `git push` dessa mesma branch, basta digitar `git push` sem mais nada.

Depois do `push`, abra o repositório no site do GitHub: costuma aparecer um aviso amarelo com o botão **Compare & pull request** — clique nele, confira o título e clique em **Create pull request**. Acompanhe a aba **Actions**: a pipeline deve rodar até o `terraform plan` e parar aí, sem aplicar. Só depois de revisar, clique em **Merge pull request** na página do PR — é esse merge, que junta a branch na `main`, que dispara a execução completa com o `apply`.

3. **Quebrando a formatação de propósito:** desalinhe a indentação de um bloco no `main.tf` e faça o `push`. Explique por que o job falha na etapa `terraform fmt -check` antes mesmo de chegar ao `plan`. Qual o valor de "falhar cedo" (fail fast) em uma esteira de CI/CD?

4. **Secrets e variáveis sensíveis:** pesquise como o GitHub Actions permite guardar segredos (aba *Settings* > *Secrets*) e reescreva o passo `Terraform Apply` supondo que seu provedor exigisse uma credencial de nuvem (ex.: `TF_VAR_aws_access_key`). Por que nunca se deve colocar uma senha ou chave direto no `main.tf` versionado — como a `POSTGRES_PASSWORD` que você usou "no bruto" no Lab 1?

5. **Rollback:** simule um incidente causado por uma mudança ruim (por exemplo, você aplicou uma porta que conflita com outro serviço). Reverta o commit problemático (`git revert`) e observe a pipeline reaplicar automaticamente o estado anterior. Que vantagem isso traz em relação a corrigir manualmente em produção?

6. **Estado efêmero do banco:** rode a pipeline duas vezes seguidas e, na primeira execução, insira um dado no banco via `psql` (dentro de um passo extra do workflow, se quiser experimentar). Confirme que, na segunda execução, esse dado não está mais lá. Explique por que isso aconteceria mesmo se o `docker_volume.db_data` do Lab 1 estivesse declarado no `main.tf`.

7. **Atualizando o site pela pipeline:** edite o `site/index.html` e observe a automação recriar o site com o conteúdo novo — sem você tocar em `terraform apply`.

   Por que isso não é tão simples quanto parece: o container Nginx que a pipeline cria vive dentro do runner do GitHub Actions, uma máquina temporária que é apagada assim que a execução termina (o mesmo motivo pelo qual os dados do banco não sobrevivem, explicado na nota da seção 3). Isso significa que, ao contrário do Nginx que você criou manualmente na sua VM no Lab 1, **você não vai conseguir abrir o navegador depois e visitar esse site** — ele já não existe mais quando a pipeline termina. Para provar que a automação funcionou, a verificação precisa acontecer *dentro* da própria execução, enquanto o container ainda está de pé.

   Passo 1 — Faça a mudança no site. No VS Code, edite o `site/index.html` e adicione um texto fácil de identificar depois — por exemplo, uma linha nova como `<p>Atualizado via pipeline</p>`. Salve o arquivo.

   Passo 2 — Envie a mudança:

```
git add site/index.html

git commit -m "Atualiza conteúdo do site"

git push
```

   Passo 3 — Ensine a pipeline a conferir o conteúdo, não só se o container está de pé. Hoje o passo *Smoke test* do `terraform.yml` só roda `docker ps`, que confirma que o container existe, mas não olha o que ele está servindo. Edite esse passo para também usar o comando `curl` (que baixa o conteúdo de uma página, como se fosse um navegador sem tela) e o `grep` (que procura um texto dentro desse conteúdo, e falha — interrompendo o passo — se não encontrar):

```
    - name: Smoke test (verificação pós-deploy)
      if: github.ref == 'refs/heads/main' && github.event_name == 'push'
      run: |
        sleep 5
        docker ps
        curl -s http://localhost:8080 | grep "Atualizado via pipeline"

Troque `8080` pela porta que o Nginx expõe no `main.tf` de vocês, e o texto
do `grep` pelo trecho exato que você adicionou no Passo 1.
```

   Passo 4 — Envie essa mudança no workflow e acompanhe:

```
git add .github/workflows/terraform.yml

git commit -m "Smoke test confere conteúdo do site"
```

```
git push
```

Acompanhe a aba **Actions**. Se o texto novo aparecer no HTML servido pelo container, o `grep` encontra a linha e o passo passa (verde). Se você errar o texto ou esquecer de fazer o `push` do `index.html`, o `grep` não encontra nada, devolve erro, e o passo falha (vermelho) — é a pipeline provando, com um teste automático, que o conteúdo certo foi de fato publicado, e não só que "algo rodou sem erro".

Pergunta para responder: por que não basta abrir o navegador para conferir essa mudança, como fazíamos no Lab 1? O que isso ensina sobre a diferença entre testar manualmente e ter uma verificação automatizada dentro da própria pipeline?

6. Encerramento do Lab 2

Neste laboratório, a esteira que antes dependia de você digitar comandos passou a ser executada por uma pipeline de CI/CD: toda mudança agora é automaticamente formatada, validada, planejada e — só depois de revisada em um Pull Request e mesclada em `main` — aplicada. Você também viu, na prática, uma limitação importante de runners efêmeros para serviços com estado. No Lab 3, essa mesma infraestrutura ganhará instrumentação: métricas, um dashboard e um exercício de resposta a incidente, fechando os três grandes desafios da rotina de um profissional DevOps.

7. Para o entregável final

O que este Lab 2 contribui para o pacote entregue na aula de fechamento (valendo nota da P1):

- O link do seu repositório no GitHub, contendo `main.tf`, `site/` e `.github/workflows/terraform.yml`.
- Um print mostrando pelo menos uma execução **verde** (bem-sucedida) na aba Actions.
- Um print do exercício 2 (Pull Request como gate), mostrando o `plan` rodando no PR sem o `apply` ser executado.

Esse repositório é a base sobre a qual o Lab 3 vai continuar — mantenha-o atualizado.